Sistema de Funeraria

Aluno: William Inkari Huaman Lazo

FLUXO

[1. Navegador / Frontend]

(O utilizador preenche os dados no formulário HTML/JS)

▼

[2. Requisição HTTP (POST / PUT / GET)]

(Envio dos dados via API REST)

▼

[3. Servidor Bun (servidor.ts)]

(Processa a rota dinâmica e valida os dados)

▼

[4. Banco de Dados SQLite (funeraria.db)]

(Salva, atualiza ou consulta nas tabelas: Clientes, Falecidos, Atendimentos)

▼

[5. Resposta JSON & Atualização Automática]

(O servidor devolve a confirmação e o frontend atualiza a interface)

Visão Geral do Sistema

Arquivos html

- Clientes
- Falecidos
- Atendimentos

Arquitetura:

Frontend (HTML + CSS + JS) ↔
Backend (Bun + SQLite)
Comunicação via API REST

Frontend

Páginas principais:

- index.html → Dashboard
- clientes.html → Cadastro e listagem
- falecidos.html
- atendimentos.html

main.js: busca dados, preenche listas, permite editar e excluir

```
async function enviar(tipo, dados) {
    let resposta = await fetch("/api/" + tipo, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(dados)
    });
    let resultado = await resposta.json();
    alert(resultado.mensagem);
}
```

Frontend

Clientes

Titular Responsável

Contém id, nome, telefone e e-mail de contato do responsável legal.

Falecidos

Vínculo Familiar

Registra datas vitais e aponta para a chave do cliente contratante.

Atendimentos

Ordem de Serviço

Conecta cliente e falecido ao serviço, data, hora e observações.

Backend e API







Runtime Bun

Execução ultrarrápida com servidor HTTP nativo e roteador de baixíssima latência.

SQLite Embutido

Persistência relacional síncrona sem necessidade de serviço de banco externo.

Frontend Nativo

HTML5, CSS3 e JS sem dependências pesadas, garantindo interface leve e rápida.

Banco de dados

Arquivo: funeraria.db (criado automaticamente no db.ts)

Tabelas:

clientes

id,
nome,
telefone,
email

Falecidos

id,
nome,
nascimento,
falecimento,
cliente,
status

atendimentos

id,
cliente,
falecido,
serviço,
data,
hora,
status,
observação

Backend e API

Servidor Bun (servidor.ts)

Rotas da API:

- GET /api/clientes | falecidos | atendimentos
- POST /api/... → cadastrar
- PUT /api/.../id → atualizar
- DELETE /api/.../id → excluir

Também serve os arquivos estáticos (HTML, CSS, JS)

Rota 1: Redirecionamento & Arquivos Estáticos

`"/" & fetch(req)`

Redireciona a raiz para e serve arquivos estáticos (HTML, CSS, JS) dinamicamente.

Regra de Validação:
Verifica existência via Bun.file(). Retorna HTTP 404 se o arquivo não existir.

Benefício Prático

- Servidor web e API no mesmo processo.
- Sem necessidade de Nginx ou Apache.
- Entrega otimizada direto da memória.

Backend e API

Rota 2: GET /api/:tipo (Listagem Geral)

`:tipo`

Consulta Dinâmica

Lê o parâmetro de rota e executa um SELECT * na tabela correspondente.

Retorna array JSON completo para alimentar o Dashboard e telas de cadastro.

```
// Exemplo de Chamada
GET /api/clientes

// Resposta do Servidor
[ { id: 1, nome: "João", ... } ]
```

Backend e API

Rota 3: POST /api/:tipo (Inclusão)

CLIENTES

Insert Cliente

Insere nome, telefone e e-mail no banco SQLite.

FALECIDOS

Insert Falecido

Grava datas vitais, status e ID do cliente vinculado.

ATENDIMENTOS

Insert Atendimento

Registra serviço, data, hora, observação e relacioamentos.

Rota 4: PUT /api/:tipo/:id (Edição)

Atualização Direcionada

Recebe o ID do registro via parâmetro de URL e o corpo JSON atualizado.

Executa a cláusula UPDATE ... WHERE id = $id garantindo integridade e alteração pontual do registro.



Mapeamento Seguro

Parâmetros identificados com prefixo $ no SQLite previnem perda de dados e vazamentos SQL.

Rota 5: DELETE /api/:tipo/:id (Exclusão)

Remoção Física

Exclui a linha correspondente diretamente na tabela especificada pelo parâmetro :tipo.

```
DELETE FROM ${tipo} WHERE id = $id
```



Confirmação de Segurança

No frontend (`main.js`), a exclusão só ocorre após a validação explícita do operador via alerta de confirmação.